CONTEXTO MESTRE — ECOMMERCE ENGINE

Versão: 1.0

Data: 10/10/2026

Status: Planejamento consolidado — desenvolvimento ainda não iniciado

---

1. Visão geral

O objetivo é desenvolver uma plataforma própria de e-commerce com arquitetura profissional, modular, segura e escalável.

O primeiro e-commerce será inspirado visualmente na Casas Bahia, utilizando o Figma já iniciado como referência.

Figma de referência:

https://www.figma.com/design/hLukXlyPocsB7orh4aGzgM/Casasb?node-id=0-1

O objetivo não é simplesmente criar uma landing page ou uma loja com produtos estáticos. Devemos desenvolver um sistema real de comércio eletrônico, incluindo:

- Loja pública completa.
- Painel administrativo independente.
- Backend para gerenciamento e transações.
- Catálogo de produtos com variantes.
- Mídias armazenadas externamente.
- Sistema de promoções e preços.
- Carrinho, frete, checkout e pagamentos.
- Pedidos, clientes e autenticação.
- Integração para importação automatizada de produtos.

- Monitoramento, auditoria, segurança e backups.

O motor deve poder ser reutilizado para desenvolver outras lojas posteriormente.

O projeto inicial terá estrutura de multi-loja preparada, mas não desenvolverá um marketplace com múltiplos vendedores independentes neste primeiro momento.

O design terá finalidade de estudo e desenvolvimento. A publicação comercial deverá utilizar identidade e materiais próprios ou devidamente licenciados, sem representar falsamente uma marca de terceiros.

---

2. Regra principal: desenvolvimento local primeiro

Todo o desenvolvimento funcional deverá ser realizado e validado localmente antes de qualquer implantação definitiva nas VPS de produção.

Não contratar, configurar nem depender das VPS Contabo para iniciar o projeto.

Fluxo obrigatório:

1. Criar arquitetura e monorepo.
2. Preparar ambiente local.
3. Desenvolver backend e banco.
4. Implementar painel administrativo.
5. Desenvolver frontend e checkout.
6. Integrar pagamentos em sandbox.
7. Executar todos os testes necessários.
8. Validar backups e recuperação.
9. Preparar containers e configurações de produção.
10. Somente então iniciar a implantação nas VPS.

O código será independente do provedor de hospedagem.

Todas as configurações externas deverão utilizar variáveis de ambiente, sem URLs, senhas ou chaves fixadas no código.

---

3. Stack tecnológica

Monorepo

- pnpm Workspaces.
- Turborepo.
- TypeScript.
- Node.js LTS compatível.
- ESLint e Prettier.
- Scripts PowerShell para Windows.

Frontend da loja

- Next.js com App Router.
- React e TypeScript.
- CSS responsivo.
- Componentes reutilizáveis.
- Renderização otimizada para SEO.
- Uso seletivo de Server Components e Client Components.
- Otimização de imagens e cache.

Painel administrativo

- Next.js com App Router.
- Design próprio, responsivo.
- Sistema de autenticação e permissões.
- Formulários validados.
- Tabelas, filtros, busca e operações em lote.

Backend

- NestJS.
- API REST versionada.
- Contratos compartilhados.

- Zod para validações de fronteira.
- Módulos desacoplados.

Banco

- PostgreSQL.
- Prisma.
- Migrações versionadas.
- Relacionamentos e índices planejados.

Processos assíncronos

- Redis.
- BullMQ.
- Worker independente.
- Outbox transacional para eventos críticos.

Mídias

- Cloudflare R2.
- Camada de armazenamento compatível com S3.
- Miniaturas e variantes otimizadas.
- Deduplicação por hash quando possível.

Testes

- Testes unitários.
- Testes de integração.
- Testes E2E com Playwright.
- Testes de carga.
- Smoke tests.
- Validação de lint, tipos e build.

As versões específicas serão selecionadas e fixadas no início, garantindo compatibilidade entre ferramentas.

---

4. Estrutura do monorepo

Nome sugerido:

`ecommerce-engine`

Diretório Windows:

`C:\Projetos\ecommerce-engine`

Estrutura:

```
ecommerce-engine/
├── apps/
│   ├── storefront/
│   ├── admin/
│   ├── api/
│   └── worker/
│
├── packages/
│   ├── database/
│   ├── contracts/
│   ├── validation/
│   ├── auth/
│   ├── catalog/
│   ├── commerce/
│   ├── payments/
│   ├── media/
│   ├── ui/
│   ├── config/
│   └── observability/
│
├── infrastructure/
│   ├── local/
│   ├── docker/
│   ├── nginx/
│   ├── cloudflare/
│   └── deploy/
│
├── scripts/
├── docs/
├── tests/
├── pnpm-workspace.yaml
├── turbo.json
└── package.json
```

Cada aplicação deverá possuir comandos independentes para desenvolvimento, build e execução.

As aplicações devem compartilhar contratos, sem criar dependências circulares entre pacotes.

---

5. Ambientes

Desenvolvimento local

Executar simultaneamente:

- Storefront: porta 3000.
- Admin: porta 3001.
- API: porta 4000.
- Worker: processo independente.
- Redis: Docker local.
- PostgreSQL: Railway Dev ou Docker local.

Usar endereços independentes:

- `store.localhost:3000`
- `admin.localhost:3001`
- `api.localhost:4000`

Documentar a configuração necessária no Windows e oferecer fallback por localhost e portas diferentes.

Usar ambientes de teste para gateways, e-mails e serviços externos. Os testes automatizados devem ser reproduzíveis sem depender de serviços pagos.

Banco de desenvolvimento

Railway PostgreSQL será a primeira opção de teste remoto, respeitando limites de recursos e créditos disponíveis.

Manter também suporte a PostgreSQL Docker local, inclusive para testes automatizados e situações sem conexão.

Utilizar variáveis separadas por ambiente e impedir operações destrutivas acidentais sobre bancos não locais.

Produção futura

- VPS 1 Contabo: somente storefront público.
- VPS 2 Contabo: API, Admin, Worker e Redis.
- PostgreSQL: provedor gerenciado externo.
- Cloudflare R2: armazenamento de mídias.
- Cloudflare: DNS, HTTPS, CDN e proteção.

A contratação e implantação acontecerão apenas após homologação local.

---

6. Arquitetura de segurança e isolamento

O frontend público não terá acesso direto ao PostgreSQL nem às credenciais administrativas.

Toda operação de produtos, pagamentos, pedidos e clientes deverá passar por APIs autorizadas.

Separar endpoints públicos, endpoints autenticados de clientes, endpoints administrativos e endpoints exclusivos de integração.

Os serviços deverão utilizar:

- HTTPS.
- Cookies seguros e sessões controladas.
- MFA para administradores.
- RBAC.
- Isolamento de dados por loja.
- Proteção contra CSRF, XSS, SSRF e injeção.
- Rate limiting.
- Auditoria.
- Validação de dados no backend.

- Gestão segura de segredos.
- Verificação de webhooks.
- Proteções para importação de URLs externas.

Nenhuma credencial deve ser armazenada no frontend ou enviada ao Git.

---

7. Arquitetura de catálogo

O catálogo deve permitir múltiplos produtos, variantes e atributos específicos por categoria.

Entidades principais previstas:

- Store e StoreDomain.
- Brand e Category.
- Product.
- ProductVariant.
- AttributeDefinition e AttributeValue.
- MediaAsset e ProductMedia.
- Inventory e InventoryMovement.
- Price e PriceHistory.
- Promotion e PromotionTarget.
- ImportBatch e ImportItem.

Os nomes e relacionamentos definitivos serão definidos na modelagem Prisma.

Produto principal

Armazena informações comuns:

- Nome.
- Marca.
- Descrição.
- Categoria.

- Características técnicas compartilhadas.
- SEO.
- Galeria geral.
- Status de publicação.

Variantes

Armazenam informações que podem mudar:

- Cor.
- Armazenamento.
- Tamanho.
- Voltagem.
- Capacidade.
- SKU.
- Preço.
- Estoque.
- Peso e dimensões.
- Imagens específicas.

Não duplicar imagens idênticas entre variantes.

A mesma mídia poderá ser reutilizada mediante referências a um objeto armazenado uma única vez.

O sistema precisa reconhecer quando produtos parecidos são modelos diferentes, evitando agrupamentos incorretos.

Escala

Primeira meta comercial ilustrativa: 10 mil a 50 mil produtos principais.

A arquitetura deverá ser testada com catálogos significativamente maiores, incluindo centenas de milhares de SKUs, sem pressupor capacidade de produção antes dos benchmarks.

---

8. Categorias e estrutura comercial

O painel poderá cadastrar departamentos, categorias, subcategorias e atributos.

Exemplos:

Telefonia → Smartphones → Apple.

Casa → Eletrodomésticos → Geladeiras.

Móveis → Sala → Sofás.

Cada categoria poderá definir atributos relevantes, filtros, ordenação, imagens de capa e regras de exibição.

Ao alterar uma categoria, o frontend deverá refletir a atualização através da API e dos mecanismos de invalidação de cache.

---

9. Armazenamento e processamento de mídia

Não armazenar imagens de produtos dentro do código-fonte, do diretório público do Next.js ou no disco permanente das VPS.

Criar um módulo responsável por:

1. Receber upload ou URL autorizada.
2. Validar origem, tipo e tamanho.
3. Processar de forma segura.
4. Detectar duplicações por hash.
5. Transferir ao armazenamento de objetos.
6. Criar versões otimizadas quando necessário.
7. Registrar metadados no PostgreSQL.
8. Associar mídias aos produtos e variantes.

Permitir imagens e vídeos, com limites e políticas específicas.

O backend deverá controlar as referências e autorizações. O navegador consumirá URLs adequadas de entrega, sem acesso a credenciais internas.

---

10. Importação automatizada de produtos

Será criado futuramente um segundo monorepo chamado provisoriamente `product-collector`.

Esse sistema não faz parte do código principal do e-commerce.

Sua função será obter dados de fontes autorizadas, normalizá-los e enviá-los para lojas específicas.

O e-commerce atual deve oferecer a interface de recebimento desde o início.

Fluxo

Coletor externo → JSON → API de importação → Validação → BullMQ → PostgreSQL e R2 → Atualização do catálogo.

Contrato de integração

O JSON deverá conter:

- Versão do contrato.
- Identificação da loja de destino.
- Fonte dos dados.
- ID único do lote.
- Identificador externo por produto.
- Identificador de agrupamento.
- Nome, descrição e marca.
- Categoria e atributos.
- Variantes e SKUs.
- Preços em centavos.
- Estoque.
- URLs de imagens e vídeos.

O sistema deverá aceitar lotes, acompanhar processamento e retornar erros estruturados.

Requisitos obrigatórios

- Autenticação máquina a máquina.
- Chaves restritas por loja.
- Rate limiting.
- Idempotência.
- Deduplicação por origem e identificador externo.
- Validação de esquema JSON.
- Limites de tamanho.
- Processamento assíncrono.
- Relatórios de erros.
- Reprocessamento.
- Política para conflitos entre dados importados e alterações manuais.
- Prevenção de requisições inseguras durante a obtenção de imagens.
- Registro da procedência e autorização de uso dos dados.

Reservar uma API `/v1/integrations/catalog/` para essas operações.

O coletor não precisará gerar ZIP nem baixar imagens localmente. A transferência de mídia será feita diretamente pelos serviços de backend.

---

11. Sistema de promoções

O painel deverá permitir:

- Desconto percentual por produto.
- Desconto por variante.
- Desconto por categoria.
- Desconto global da loja.
- Períodos de vigência.
- Ativação e desativação.

- Histórico de preços.
- Exibição dos descontos nos cards.

O usuário poderá informar livremente percentuais válidos entre 0 e 100, incluindo casas decimais.

Adotar uma política documentada para conflitos entre promoções e evitar descontos cumulativos involuntários.

Calcular preços utilizando representação decimal ou unidades monetárias inteiras, nunca aritmética financeira baseada em números de ponto flutuante sem controle.

O checkout deverá recalcular os valores no servidor.

---

12. Checkout e pagamentos

Implementar:

- Carrinho persistente.
- Endereços.
- Consulta e cálculo de frete.
- Cupons.
- Revisão do pedido.
- Seleção de pagamento.
- Criação de transação.
- Confirmação por webhook autenticado.
- Histórico de pedidos.
- Cancelamentos e estornos.
- Tratamento de pagamentos pendentes e falhos.

Criar uma camada de adaptadores de gateways.

O painel permitirá selecionar entre os provedores implementados no backend e configurados para cada loja.

Nunca armazenar dados completos de cartão. Utilizar integração compatível com os requisitos do provedor e as obrigações de segurança aplicáveis.

Implementar idempotência e tratamento correto de concorrência de estoque, pedidos e pagamentos.

Usar exclusivamente credenciais de sandbox durante o desenvolvimento.

---

13. Painel administrativo

O painel deverá disponibilizar:

Dashboard: receitas, pedidos, indicadores, produtos mais vendidos.

Produtos: CRUD, variantes, mídia, preços, estoque e publicação.

Importação: envio de JSON, acompanhamento de lotes, erros e reprocessamento.

Categorias: hierarquia, atributos, filtros e organização.

Promoções: descontos e campanhas.

Vitrine: banners, seções, produtos destacados, organização e SEO.

Clientes: cadastro, histórico, endereços e preferências.

Pedidos: status, pagamentos, fretes, cancelamentos e devoluções.

Gateways: seleção de provedor instalado, configuração segura e teste de conexão.

Marketing: campanhas, e-mails, cupons, segmentação e preferências de contato.

Segurança: usuários administrativos, permissões, histórico e auditoria.

O sistema deve ser utilizável por operadores sem precisar editar código.

---

14. Frontend público

A implementação visual terá como primeira referência o frame Hero do Figma, com dimensões 1920 × 1024.

Construir a interface com componentes adaptáveis, não com posições fixas para uma resolução específica.

Funcionalidades esperadas:

- Header e pesquisa.
- Departamentos.
- Banners promocionais.
- Listagens e carrosséis de produtos.
- Categorias.
- Busca e filtros.
- Página de produto.
- Seleção de variantes.
- Favoritos.
- Carrinho.
- Cadastro e login.
- Checkout.
- Pedidos.
- Conta do cliente.
- Páginas institucionais necessárias.

Todo conteúdo de catálogo será carregado dinamicamente.

A home poderá começar com dados demonstrativos enquanto o backend é desenvolvido. Depois, os mesmos componentes serão conectados às APIs definitivas.

Preparar SEO, acessibilidade, responsividade, loading states, tratamento de erros e otimização de desempenho.

---

15. Clientes e marketing

O banco armazenará apenas os dados necessários à operação e às finalidades autorizadas.

Criar entidades e fluxos para clientes, endereços, pedidos, preferências e histórico comercial.

Planejar integrações futuras com provedores de e-mail e mensageria.

O remarketing deverá respeitar a LGPD e as preferências de comunicação, incluindo gestão de consentimento quando exigido, finalidade, retenção, descadastro e mecanismos de solicitação de dados.

Não implementar disparos em massa sem controles adequados.

---

16. Backups, falhas e recuperação

Backups serão planejados desde o início.

Desenvolvimento

Manter migrações e scripts versionados.

Criar rotinas de exportação e restauração do banco, com cópias externas criptografadas quando houver dados relevantes.

Os testes automatizados não deverão utilizar dados pessoais reais.

Produção

Planejar PostgreSQL com recuperação pontual, conforme o serviço contratado.

Também implementar:

- Backups lógicos independentes.
- Retenção definida.
- Cópias de mídia em outro domínio de falha.
- Monitoramento de falhas de backup.
- Testes periódicos de restauração.
- Procedimentos documentados de recuperação.
- Reimplantação das VPS a partir de código e imagens versionadas.
- Verificação de integridade após restauração.

Redis não poderá ser a única fonte de registro de compras e pagamentos.

Criar processos de reconciliação de pagamentos e eventos críticos para lidar com falhas intermediárias.

Nenhum backup será considerado validado sem um teste de restauração bem-sucedido.

---

17. Estratégia de implantação futura

A implantação usará duas VPS Contabo:

VPS 1 — Storefront

Executará apenas os componentes necessários à vitrine pública.

VPS 2 — Principal

Executará API, painel administrativo, worker e Redis, com isolamento dos serviços por containers e redes.

O PostgreSQL ficará fora dessas VPS.

Os serviços serão empacotados para implantação reproduzível.

Preparar:

- Dockerfiles.
- Docker Compose de produção.
- Proxy reverso.
- HTTPS e Cloudflare.
- Firewall e acesso SSH seguro.
- CI/CD.
- Migrações controladas.
- Health checks.
- Logs e monitoramento.
- Deploys versionados.
- Procedimento de rollback.
- Procedimento para substituir uma VPS.

Não colocar credenciais, uploads permanentes ou dados críticos exclusivamente nos discos das VPS.

---

18. Roadmap oficial de implementação

Etapa 0 — Planejamento técnico

Formalizar arquitetura, decisões, entidades, endpoints, contratos, riscos, dependências e critérios de aceite.

Etapa 1 — Fundação do monorepo

Criar estrutura, configurações, aplicativos, pacotes, scripts PowerShell, lint, testes básicos e builds independentes.

Etapa 2 — Ambiente local e infraestrutura

Configurar Docker, PostgreSQL, Redis, variáveis, migrations e conexões.

Etapa 3 — Banco de dados e multi-loja

Implementar schema, relacionamentos, índices e isolamento por loja.

Etapa 4 — Autenticação e permissões

Implementar clientes, administradores, sessões, recuperação de acesso, MFA administrativo e RBAC.

Etapa 5 — Catálogo

Criar produtos, categorias, atributos, marcas, variantes, preços e estoques.

Etapa 6 — Mídias

Implementar camada de armazenamento, upload, importação por URL, deduplicação e otimização.

Etapa 7 — Importação JSON

Criar contratos, endpoints, filas, importações por lote, atualização e relatórios.

Etapa 8 — Promoções e preços

Implementar motor de descontos, prioridades, campanhas e histórico.

Etapa 9 — Carrinho, pedidos e frete

Construir fluxos comerciais, cálculo confiável de valores, endereços e pedidos.

Etapa 10 — Gateways de pagamento

Criar interfaces, adaptadores iniciais, configuração administrativa, sandbox, webhooks e estornos.

Etapa 11 — Painel administrativo completo

Construir todas as interfaces operacionais utilizando os endpoints disponíveis.

Etapa 12 — Loja pública completa

Concluir a interface baseada no Figma, páginas comerciais, responsividade e integração real ao backend.

O protótipo inicial do Hero poderá ser desenvolvido antecipadamente, assim que o monorepo estiver funcional, sem bloquear o desenvolvimento do backend.

Etapa 13 — Clientes e marketing

Implementar contas, históricos, preferências e infraestrutura de comunicação.

Etapa 14 — Testes integrados

Executar testes unitários, integração, E2E, concorrência, importação em volume, segurança e desempenho.

Etapa 15 — Recuperação e backups

Testar restauração do banco, recuperação de mídia, reconstrução da aplicação e falhas de serviços.

Etapa 16 — Preparação da produção

Preparar configuração final das VPS, banco gerenciado, containers, proxy, monitoramento e implantação.

Etapa 17 — Homologação e publicação

Executar migrações, smoke tests, testes de compra, verificação de segurança e publicação controlada.

Nenhuma etapa deverá ser considerada concluída somente porque o código foi escrito.

---

19. Regras de trabalho para o ChatGPT

O desenvolvimento acontecerá principalmente em Windows e PowerShell.

Sempre que criar um script:

1. Usar sintaxe correta de PowerShell.
2. Verificar caminhos Windows.
3. Evitar comandos exclusivos de Bash.
4. Validar compatibilidade com versões de Node, pnpm, Prisma, Next.js e NestJS.
5. Tratar erros e códigos de saída.
6. Preservar arquivos existentes.
7. Evitar comandos destrutivos sem necessidade.

Priorizar arquivos completos e prontos para copiar e colar.

Antes de avançar em cada etapa, executar ou instruir a execução de:

- Lint.
- Typecheck.
- Testes aplicáveis.
- Build.
- Validação de banco e migrações, quando necessário.

Nunca afirmar que um teste foi executado quando ele não foi realmente executado.

Manter documentação técnica atualizada e registrar decisões importantes.

O GitHub será utilizado para versionamento, com commits claros por etapa. Não realizar push com segredos nem artefatos locais.

Evitar mudanças não relacionadas à etapa atual.

Corrigir problemas antes de continuar quando eles bloquearem a integridade do projeto.

Não refazer partes já aprovadas sem justificativa técnica.

Não solicitar confirmações desnecessárias para decisões já estabelecidas. Perguntar somente quando faltar algo essencial ou quando houver risco significativo.

---

20. Padrão de entrega de cada etapa

Cada etapa deve fornecer:

Objetivo: o que está sendo construído.

Dependências: o que deve existir antes.

Arquivos: quais arquivos serão criados ou modificados.

Implementação: código completo e integrado.

Execução: comandos PowerShell exatos.

Validação: testes e critérios verificáveis.

Resultado: o que deve funcionar ao final.

Versionamento: proposta de commit e atualização de documentação.

Próxima etapa: dependências e continuidade.

Manter um documento `docs/CONTEXTO_MESTRE.md` e documentos de decisões e etapas.

Atualizar esses arquivos sempre que houver mudanças relevantes na arquitetura ou nas regras de funcionamento.

---

21. Primeira instrução de execução

Comece pela **Etapa 0 — Planejamento técnico**.

Analise integralmente este contexto mestre e produza:

1. Arquitetura técnica definitiva para a versão inicial.
2. Mapa dos aplicativos e pacotes do monorepo.
3. Modelo inicial de dados e relacionamentos.
4. Mapa completo dos módulos do backend.
5. Contratos e fluxo de importação JSON.
6. Estratégia de desenvolvimento local.
7. Estratégia de testes e proteção dos dados.
8. Dependências e ordem de implementação.
9. Critérios de aceite da Etapa 1.

Depois disso, siga com a fundação do monorepo, respeitando o planejamento, sem antecipar deploy em VPS e sem misturar o Product Collector externo ao projeto principal.

Objetivo final: construir e validar localmente um sistema completo e reutilizável de e-commerce, pronto para implantação posterior em duas VPS Contabo com PostgreSQL independente.